Training ConvNets

Lecturer: Dr. Samuel Kakuba

March 10 and 12, 2026

Part of this material came from Prof. Heechul Jung' Lecture Materials at the Department of Artificial Intelligence, Kyungpook National University

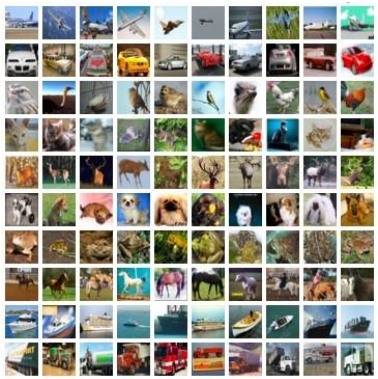
Preparation

Some slides from CS232n, Stanford University

Step 0 : Choose dataset

<MNIST>

<SVHN>



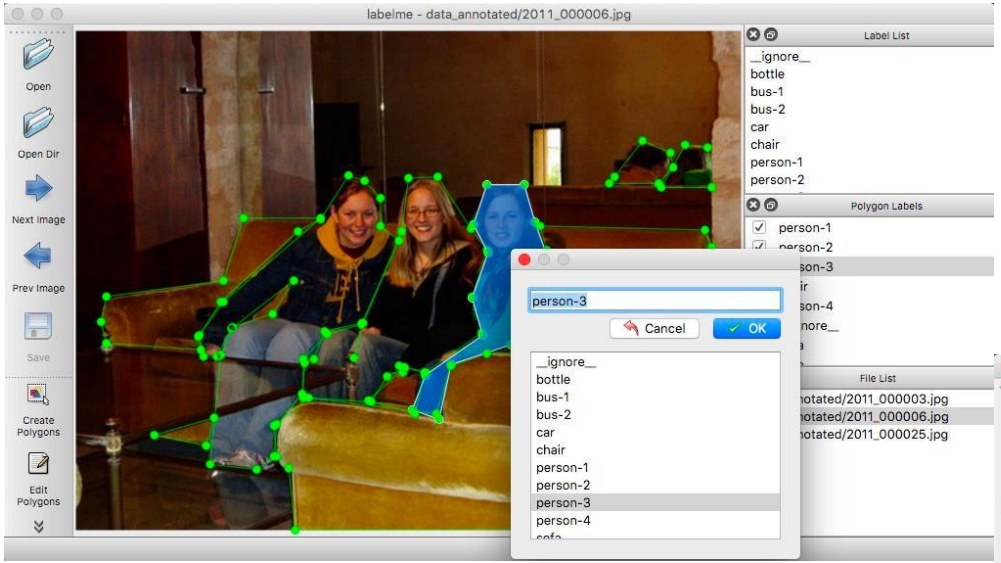
<CIFAR-10 / 100>

<Fashion MNIST>

<ImageNet Dataset>

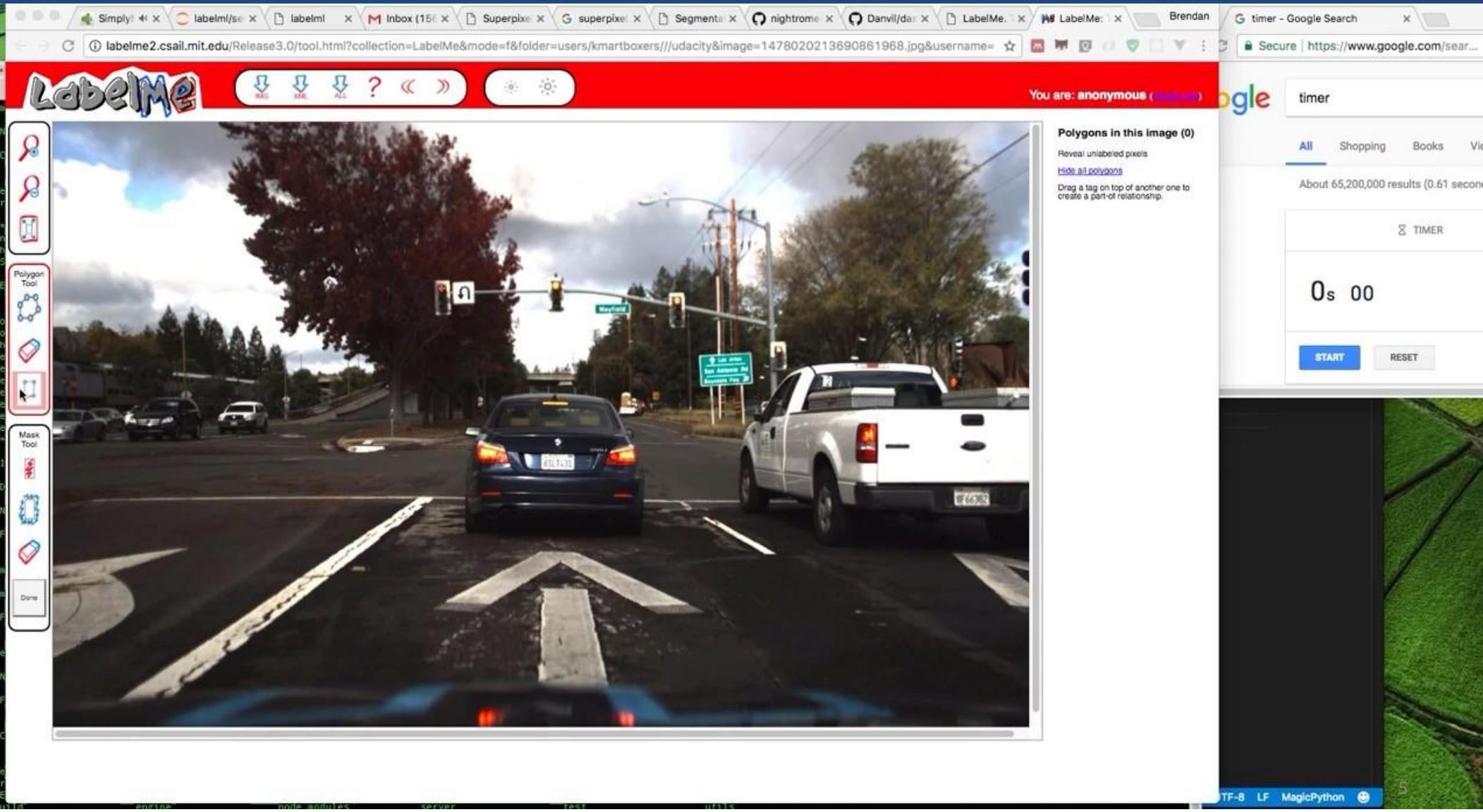
Step 0 : Build own dataset

Labelme

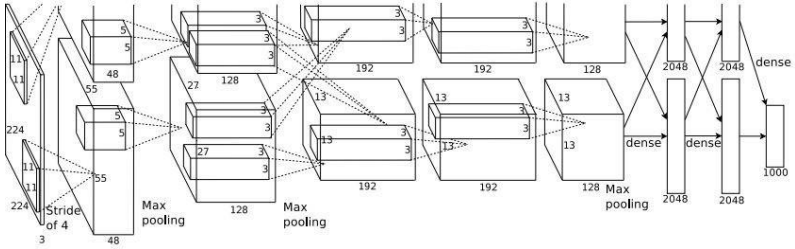


https://github.com/wkentaro/labelme

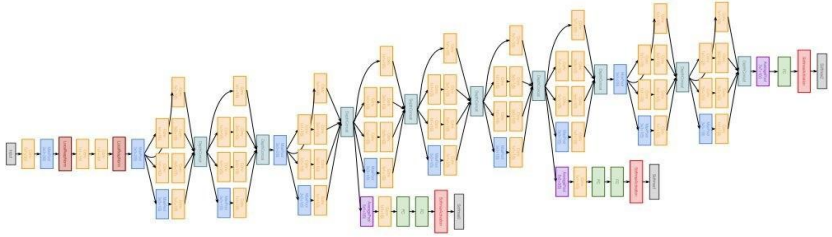
https://www.youtube.com/watch?v=OFjrDF7Jrgo


labelme2.csail.mit.edu/Release3.0/tool.html?collection=LabelMe&mode=f&folder=users/kmartboxers///udacity&image=1478020213690861968.jpg&username=
LabelMe
You are: anonymous
Polygon Tool
Mask Tool
Done
Polygons in this image (0)
Reveal unlabeled pixels
Hide all polygons
Drag a tag on top of another one to create a part-of relationship.
timer - Google Search
timer
All
Shopping
Books
About 65,200,000 results (0.61 second
TIMER
0s 00
START
RESET

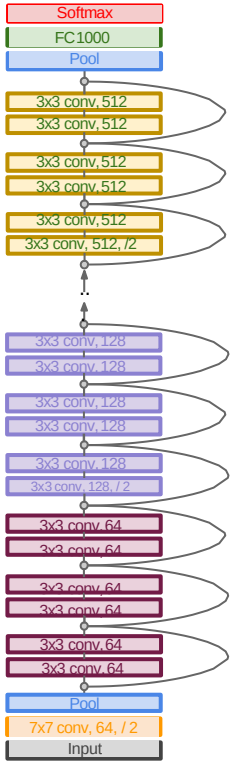
Step 1 : Choose the architecture



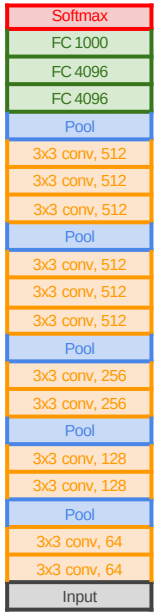
AlexNet



GoogleNet

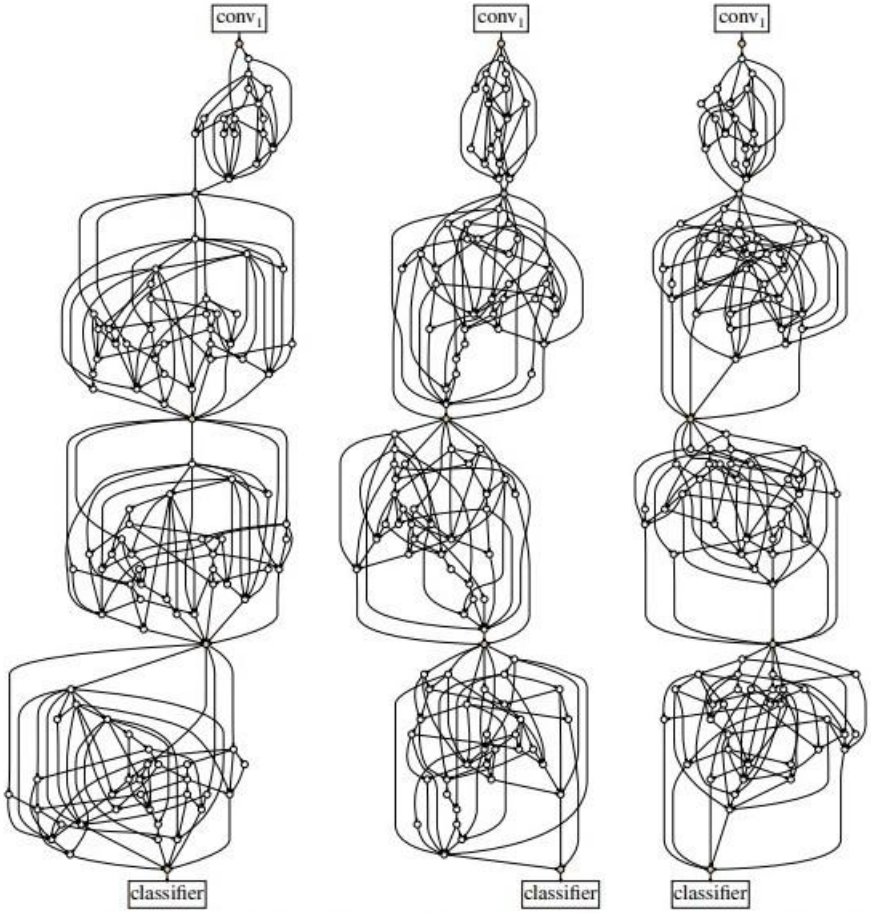


ResNet



VGG16

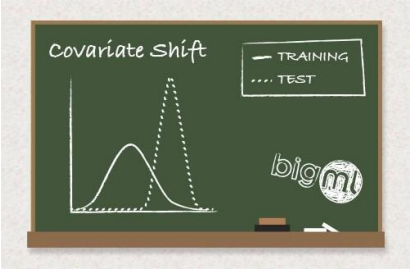
Step 1 : Build own architecture



Put Batch Normalization into your architecture

- Let's normalize all activations in a batch to have N(0,1)

$$\widehat{x}^{(k)} = \frac{x^{(k)} - \mathrm{E}[x^{(k)}]}{\sqrt{\mathrm{Var}[x^{(k)}]}}$$



- Motivation : Internal Covariate Shift?
  - Distribution of activations at the 1st epoch is severely different from that of the 10th epo ch, even though inputs are same.
  - This is due to the change in parameters during training.
  - In other words, if we normalize batch activations, then parameters do not fluctuate, thus r esulting in fast & stable training.
    - E.g. BN-x5 needs 14 times fewer steps than Inception to reach the 72.2% accuracy.

Ioffe, Sergey, and Christian Szegedy. "Batch normalization: Accelerating deep network training by reducing internal covariate shift." arXiv preprint arXiv:1 502.03167 (2015).

Batch Normalization

Normalize:

$$\boxed{\widehat{x}^{(k)} = \frac{x^{(k)} - \mathrm{E}[x^{(k)}]}{\sqrt{\mathrm{Var}[x^{(k)}]}}}$$

And then allow the network to squash the range if it wants to:

$$\boxed{y^{(k)} = \gamma^{(k)}\widehat{x}^{(k)} + \beta^{(k)}}$$

Note, the network can learn:

$$\gamma^{(k)} = \sqrt{\mathrm{Var}[x^{(k)}]}$$

$$\beta^{(k)} = \mathrm{E}[x^{(k)}]$$

to recover the identity mapping.

Fei-Fei Li & Andrej Karpathy & Justin Johnson, CS231n: Convolutional Neural Networks for Visual Recognition, Spring 2016. (Stanford University)

Batch Normalization

- Training phase

Input: Values of x over a mini-batch: $\mathcal{B} = \{x_{1\ldots m}\}$;
Parameters to be learned: γ, β

Output: $\{y_i = \mathrm{BN}_{\gamma,\beta}(x_i)\}$

$$\mu_\mathcal{B} \leftarrow \frac{1}{m}\sum_{i=1}^{m} x_i \qquad \text{// mini-batch mean}$$

$$\sigma_\mathcal{B}^2 \leftarrow \frac{1}{m}\sum_{i=1}^{m} (x_i - \mu_\mathcal{B})^2 \qquad \text{// mini-batch variance}$$

$$\widehat{x}_i \leftarrow \frac{x_i - \mu_\mathcal{B}}{\sqrt{\sigma_\mathcal{B}^2 + \epsilon}} \qquad \text{// normalize}$$

$$y_i \leftarrow \gamma \widehat{x}_i + \beta \equiv \mathrm{BN}_{\gamma,\beta}(x_i) \qquad \text{// scale and shift}$$

To speedup training with BN,

- Increase learning rate
- Remove dropout
- Reduce L2 regularization
- Accelerate the learning rate decay

Ioffe, Sergey, and Christian Szegedy. "Batch normalization: Accelerating deep network training by reducing internal covariate shift." arXiv preprint arXiv:1502.03167 (2015).

Batch Normalization

- Test phase
  - **Do not** calculate the mean and std **from test data.**
  - Use the moving (running) averages of mean and std calculated from minibatch o f training data

=> Moving average approximates mean and std calculated from full batch of training data

Step 2 : Initialize the weights

Before we try training, lets **initialize** well:

- set weights to small random numbers

```
W = 0.001* np.random.randn(D,H)
```

(Matrix of small numbers drawn randomly from a gaussian)
Warning: *This is not optimal, but simplest! (More on this later)*

- set biases to zero

Step 2: Initialize the weights

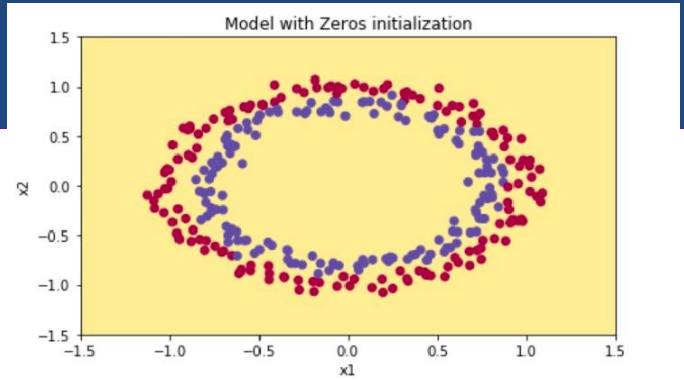
- Pitfall : all zero initialization
- Small random numbers →Small gradient.
- Use Xavier or He's initialization method

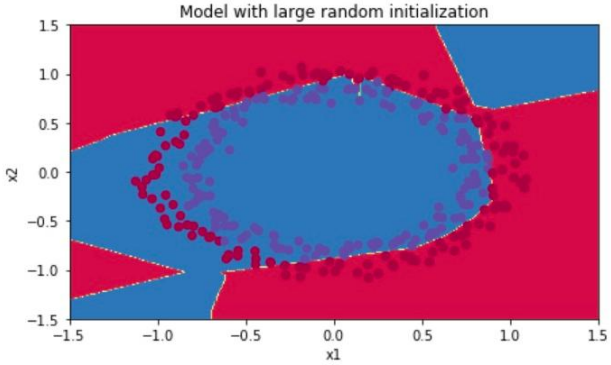
$$\sqrt{\frac{1}{size^{[l-1]}}}$$

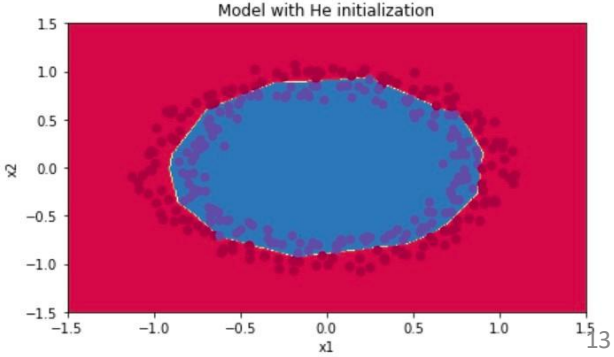
$W^{[l]}$ = *np.random.randn(size_l, size_l-1) * np.sqrt(1/size_l-1)*

$$\sqrt{\frac{2}{size^{[l-1]}}}$$

$W^{[l]}$ = *np.random.randn(size_l, size_l-1) * np.sqrt(2/size_l-1)*

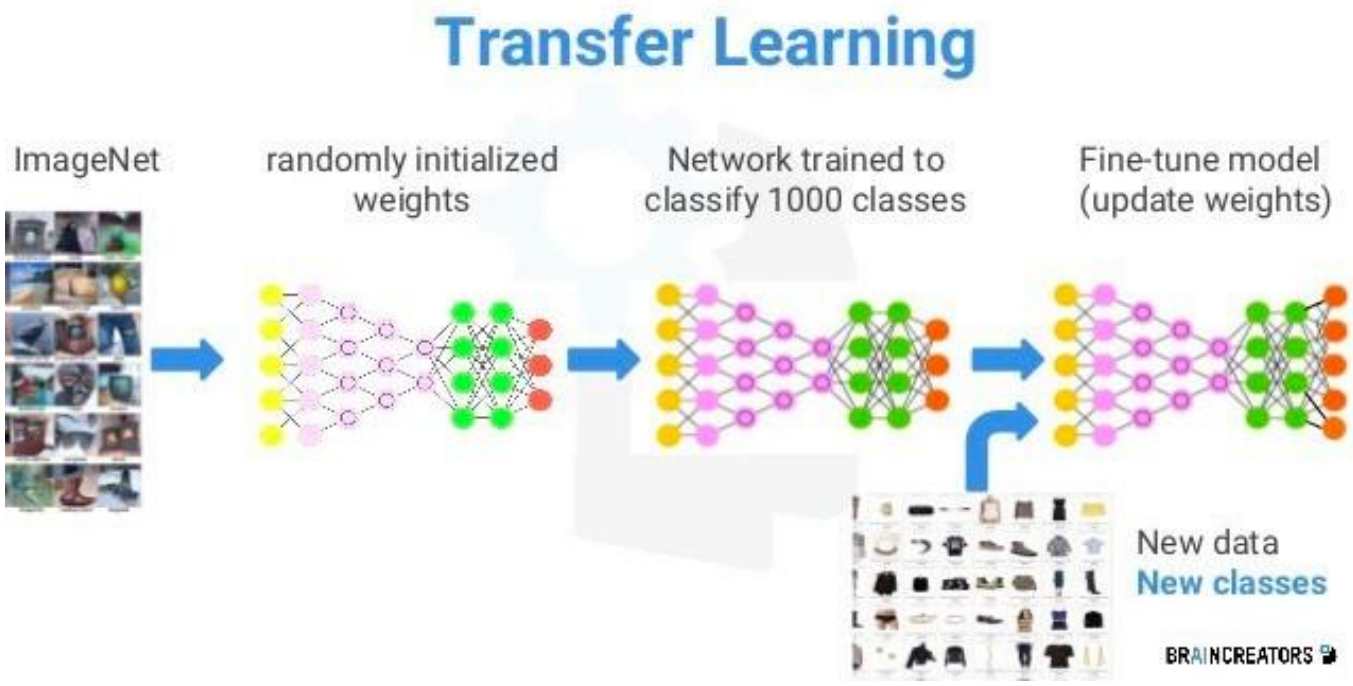






Step 2: Initialize the weights

- Use Pre-trained Model



Step 3: Choose optimizer

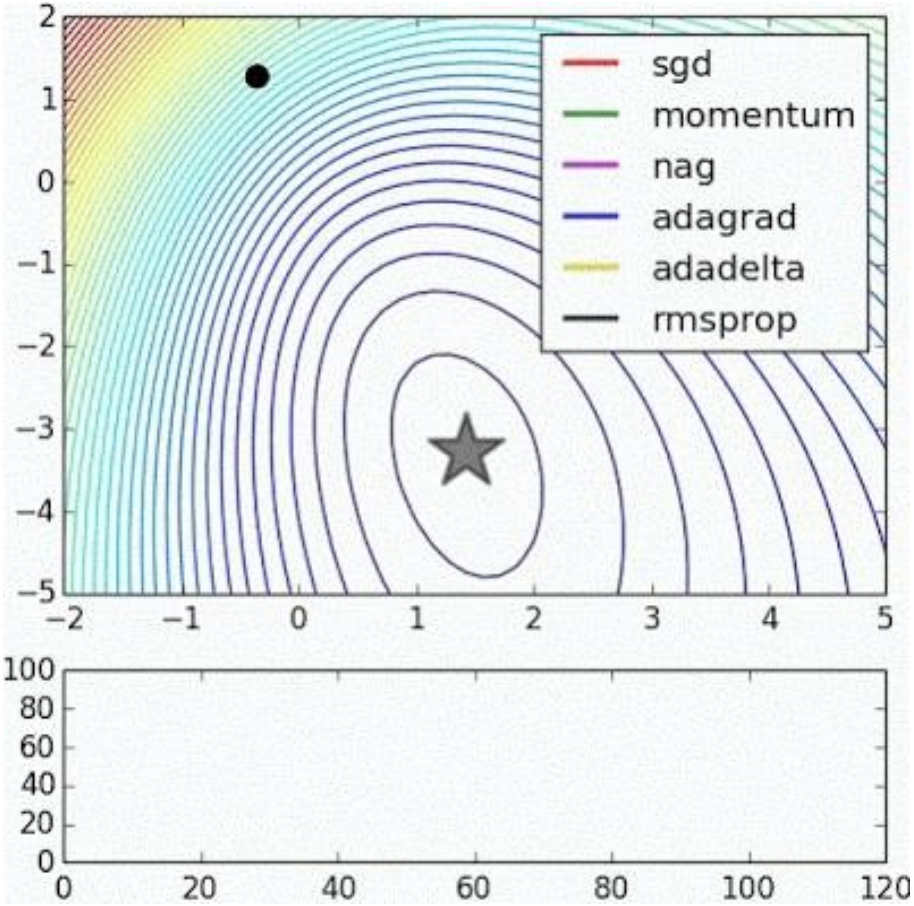


Image Credit : Alec Radford https://www.reddit.com/r/MachineLearning/co mments/2gopfa/visualizing_gradient_optimizat ion_techniques/cklhott

Saddle Point?

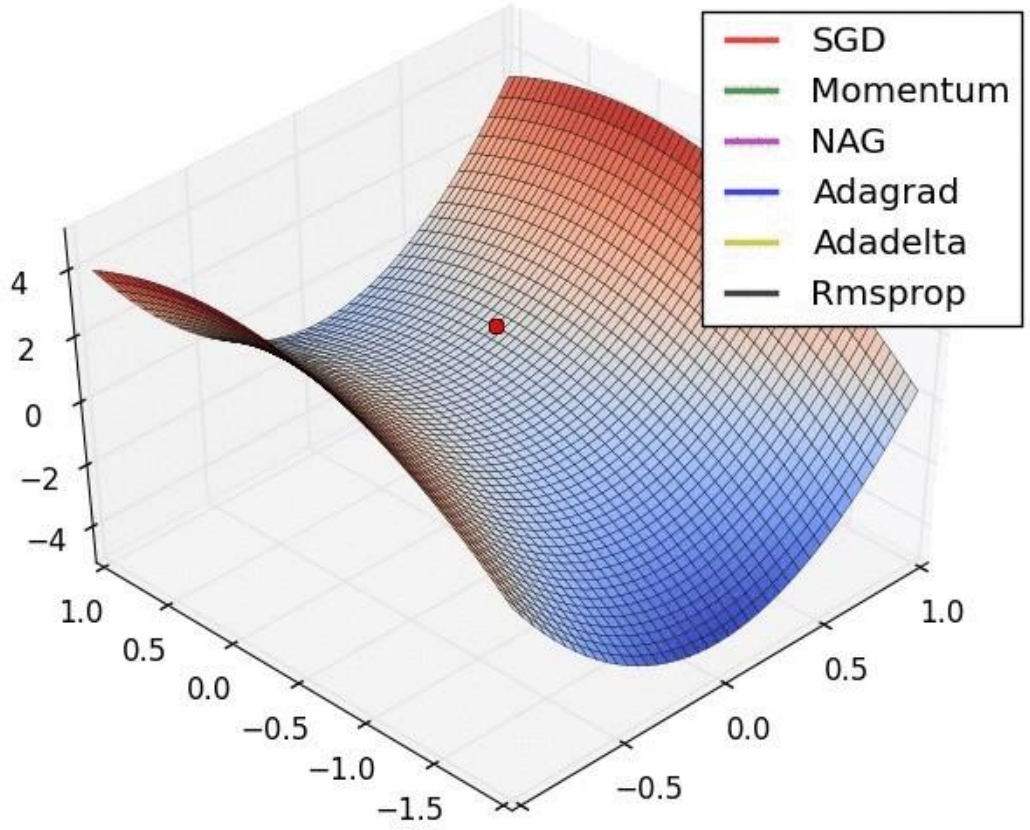


Image Credit : Alec Radford https://www.reddit.com/r/MachineLearning/comments/2gopfa/visualizing_gradient_optimization_techniques/cklhott

SGD(Stochastic Gradient Descent)

- Original SGD

$$\theta_{t+1} = \theta_t - \eta \nabla \mathcal{L}(\theta_t)$$

- SGD with Momentum

$$v_{t+1} = \mu v_t - \eta \nabla \mathcal{L}(\theta_t)$$
$$\theta_{t+1} = \theta_t + v_{t+1}$$

Equations : http://colinraffel.com/wiki/stochastic_optimization_techniques

Adagrad

- Original SGD

$$\theta_{t+1} = \theta_t - \eta \nabla \mathcal{L}(\theta_t)$$

- Adagrad

$$g_{t+1} = g_t + \nabla \mathcal{L}(\theta_t)^2$$

$$\theta_{t+1} = \theta_t - \frac{\eta \nabla \mathcal{L}(\theta_t)}{\sqrt{g_{t+1}} + \epsilon}$$

When g_t is small, then step size is large. As g_t gets larger, step size decays

g_t never decreases. Therefore, learning rates will only decay. Sometimes, this results in too fast decaying. So, don't use Adagrad.

***Preventing division by 0**

***In addition, epsilon affects initial learning rates ($g_0 = 0$)**

Duchi, John, Elad Hazan, and Yoram Singer. "Adaptive subgradient methods for online learning and stochastic optimization." The Journal of Machine Learning Research 12 (2011): 2121-2159.

RMSprop

- Original SGD

$$\theta_{t+1} = \theta_t - \eta \nabla \mathcal{L}(\theta_t)$$

- RMSprop

$$g_{t+1} = \gamma g_t + (1-\gamma)\nabla \mathcal{L}(\theta_t)^2$$

$$\theta_{t+1} = \theta_t - \frac{\eta \nabla \mathcal{L}(\theta_t)}{\sqrt{g_{t+1}} + \epsilon}$$

$g_t \approx E[\nabla L(\theta_t)]$

Now, learning rates c an both increase and decrease.

When the gradient is large, take small step size. When the gradient is small, take large step size.
Practically, this will eventually lead t o convergence.

Tieleman, Tijmen, and Geoffrey Hinton. "Lecture 6.5-rmsprop." COURSERA: Neural networks for machine learning (2012).

RMSprop

RMSprop is not from a paper. It's from Geoffrey Hinton's lecture note slides

rmsprop: A mini-batch version of rprop

- rprop is equivalent to using the gradient but also dividing by the size of the gradient.
  - The problem with mini-batch rprop is that we divide by a different number for each mini-batch. So why not force the number we divide by to be very similar for adjacent mini-batches?
- rmsprop: Keep a moving average of the squared gradient for each weight

$$MeanSquare(w, t) = 0.9\, MeanSquare(w, t-1) + 0.1 \left(\partial E / \partial w (t) \right)^2$$

- Dividing the gradient by $\sqrt{MeanSquare(w, t)}$ makes the learning work much better (Tijmen Tieleman, unpublished).

Tieleman, Tijmen, and Geoffrey Hinton. "Lecture 6.5-rmsprop." COURSERA: Neural networks for machine learning (2012).

Adam

$$g_{t+1} = \gamma g_t + (1-\gamma)\nabla\mathcal{L}(\theta_t)^2$$

$$\theta_{t+1} = \theta_t - \frac{\eta\nabla\mathcal{L}(\theta_t)}{\sqrt{g_{t+1}} + \epsilon}$$

RMSProp

[Kingma and Ba, 2014]

Adam update

(incomplete, but close)

```
# Adam
m = beta1*m + (1-beta1)*dx # update first moment
v = beta2*v + (1-beta2)*(dx**2) # update second moment
x += - learning_rate * m / (np.sqrt(v) + 1e-7)
```

momentum

RMSProp-like

Looks a bit like RMSProp with momentum

```
# RMSProp
cache = decay_rate * cache + (1 - decay_rate) * dx**2
x += - learning_rate * dx / (np.sqrt(cache) + 1e-7)
```

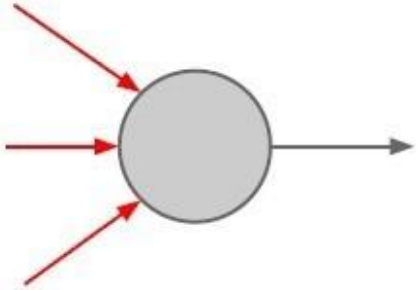
Fei-Fei Li & Andrej Karpathy & Justin Johnson, CS231n: Convolutional Neural Networks for Visual Recognition, Spring 2016. (Stanford University)

Regularization

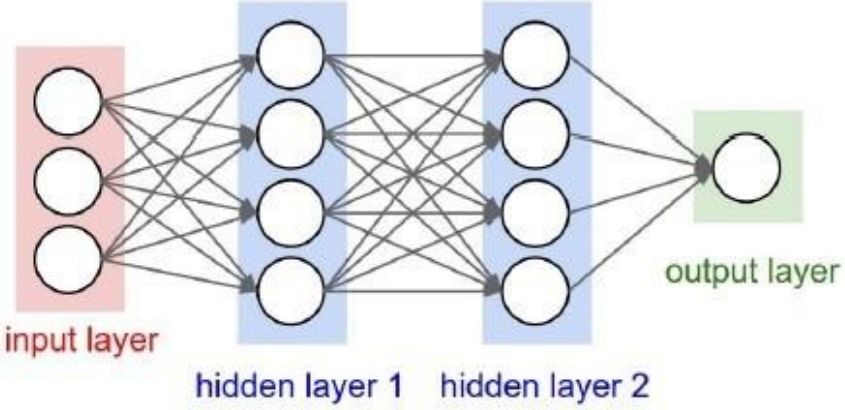
Regularization knobs

- L2 regularization $\frac{1}{2}\lambda w^2$
- L1 regularization $\lambda \mid w \mid$
- L1 + L2 can also be combined
- Max norm constraint

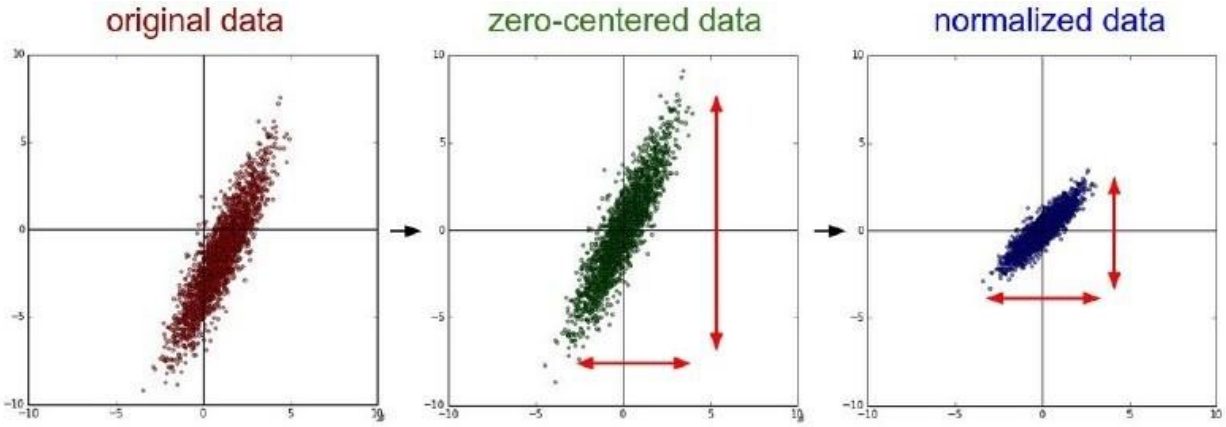
L1 is "sparsity inducing" (many weights become almost exactly zero)



enforce maximum L2 norm of the incoming weights



Step 4 : Preprocess the data



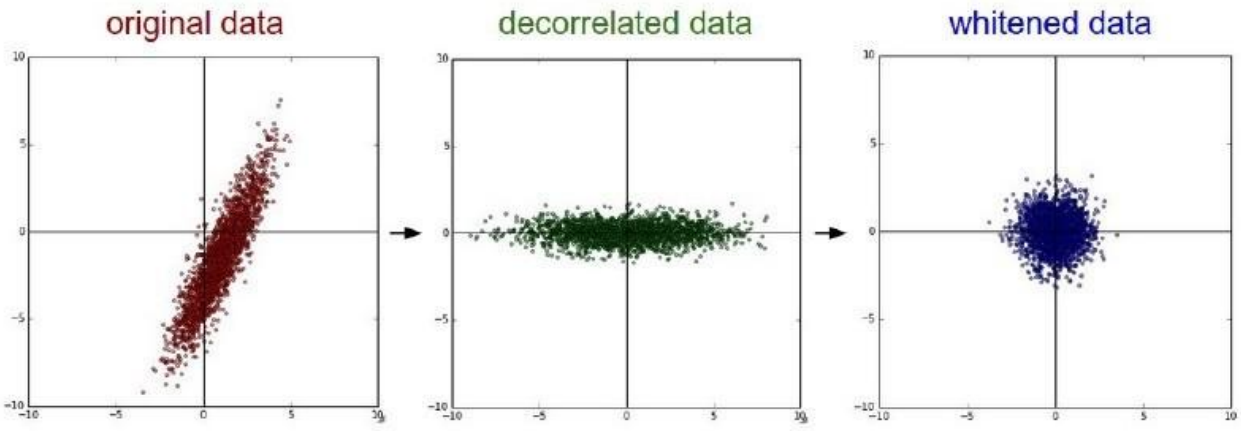

```
X -= np.mean(X, axis = 0)
```

```
X /= np.std(X, axis = 0)
```

(Assume X [NxD] is data matrix,
each example in a row)

Step 4 : Preprocess the data

In practice, you may also see **PCA** and **Whitening** of the data

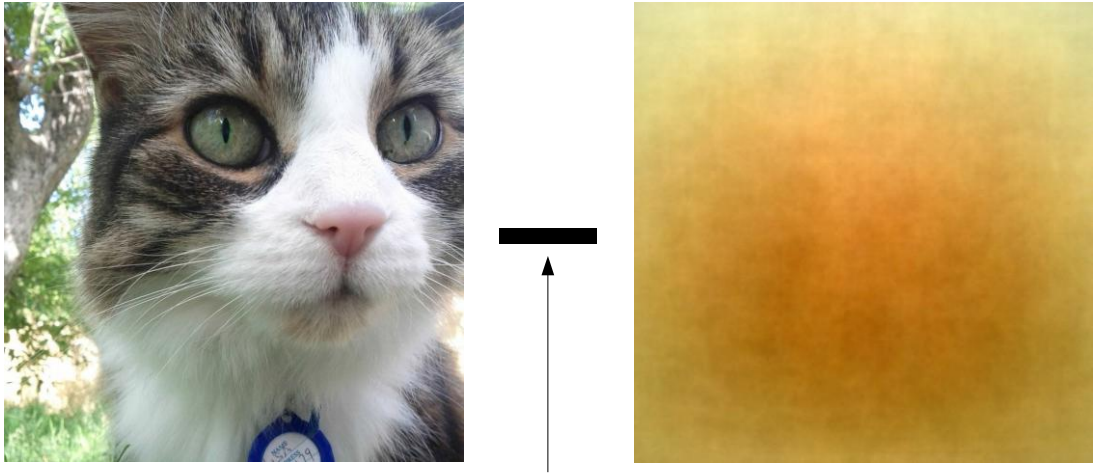


(data has diagonal covariance matrix)

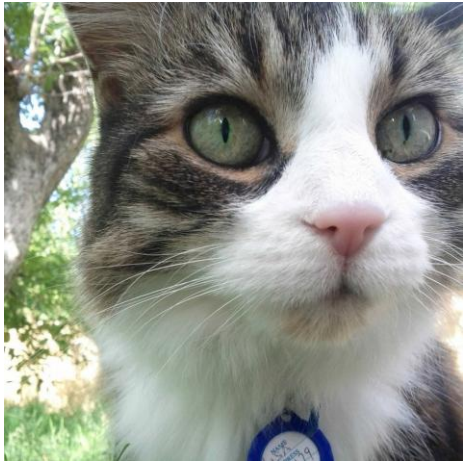
(covariance matrix is the identity matrix)

Step 4 : Preprocess the data

Centered (0-mean) RGB values.



Step 4 : Preprocess the data

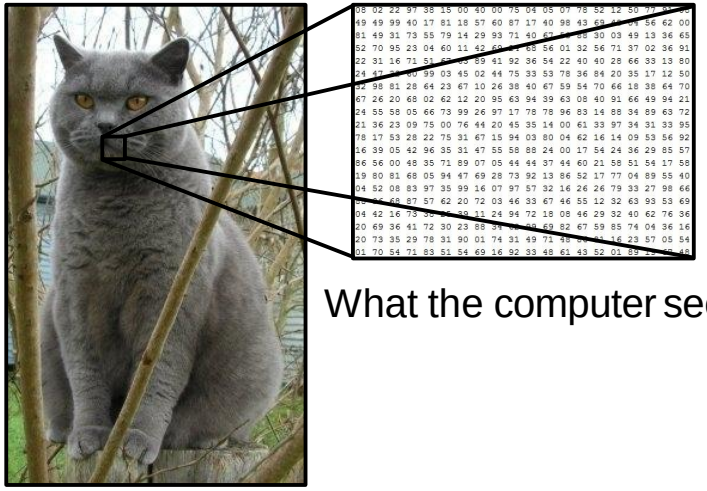


$$\left\{ \quad - \begin{pmatrix} 127.5 \\ 127.5 \\ 127.5 \end{pmatrix} \right\} \div 127.5$$

Step 4 : Preprocess the data

- i.e. simulating “fake” data

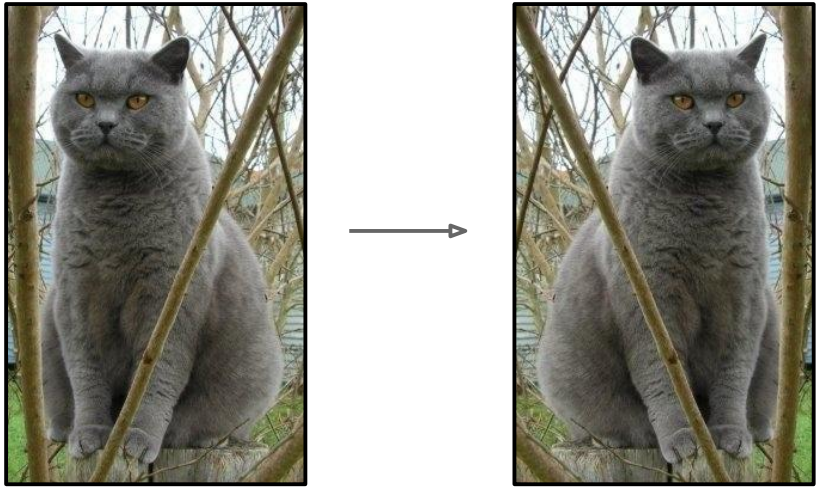
- explicitly encoding image transformations that shouldn’t change object identity.



What the computer sees

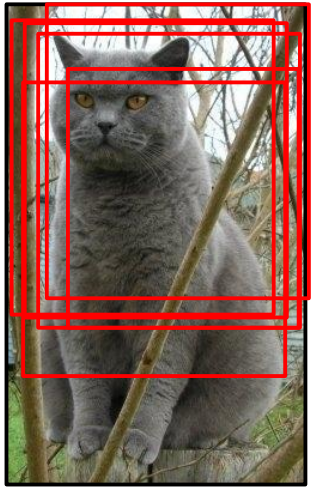
Step 4 : Preprocess the data

1. Flip horizontally



Step 4 : Preprocess the data

2. Random crops/scales



Sample these during training
(also helps a lot during test time)

e.g. common to see even up to 150 crops used

Step 4 : Preprocess the data

3. Random mix/combinations of :

- translation
- rotation
- stretching
- shearing,
- lens distortions, …(go crazy)

Step 4 : Preprocess the data

4. Color jittering

(maybe even contrast jittering, etc.)

- Simple: Change contrast small amounts, jitter the color distributions, etc.

- Vignette,... (go crazy)

Step 4 : Preprocess the data

4. Color jittering

(maybe even contrast jittering, etc.)

- Simple: Change contrast small amounts, jitter the color distributions, etc.

Fancy PCA way:

1. **Compute PCA on all [R,G,B] points values in the training data**
2. **sample some color offset along the principal components at each forward pass**
3. **add the offset to all pixels in a training image**

(As seen in *[Krizhevsky et al. 2012]*)

Run!! (Training)

Some slides from CS232n, Stanford University

Training neural network

Double check that the loss is reasonable:

```
def init_two_layer_model(input_size, hidden_size, output_size):
  # initialize a model
  model = {}
  model['W1'] = 0.0001 * np.random.randn(input_size, hidden_size)
  model['b1'] = np.zeros(hidden_size)
  model['W2'] = 0.0001 * np.random.randn(hidden_size, output_size)
  model['b2'] = np.zeros(output_size)
  return model
```

```
model = init_two_layer_model(32*32*3, 50, 10) # input size, hidden size, number of classes
loss, grad = two_layer_net(X_train, model, y_train, 0.0)
print loss
```

disable regularization

```
2.30261216167
```

loss ~2.3.
"correct " for
10 classes

returns the loss and the
gradient for all parameters

Training neural network

Double check that the loss is reasonable:

```
def init_two_layer_model(input_size, hidden_size, output_size):
  # initialize a model
  model = {}
  model['W1'] = 0.0001 * np.random.randn(input_size, hidden_size)
  model['b1'] = np.zeros(hidden_size)
  model['W2'] = 0.0001 * np.random.randn(hidden_size, output_size)
  model['b2'] = np.zeros(output_size)
  return model
```

```
model = init_two_layer_model(32*32*3, 50, 10) # input size, hidden size, number of classes
loss, grad = two_layer_net(X_train, model, y_train, 1e3)
print loss
```

crank up regularization

```
3.06859716482
```

loss went up, good. (sanity check)

Training neural network

Lets try to train now…

Tip: Make sure that you can overfit very small portion of the data

```
model = init_two_layer_model(32*32*3, 50, 10) # input size, hidden size, number of classes
trainer = ClassifierTrainer()
X_tiny = X_train[:20] # take 20 examples
y_tiny = y_train[:20]
best_model, stats = trainer.train(X_tiny, y_tiny, X_tiny, y_tiny,
                                  model, two_layer_net,
                                  num_epochs=200, reg=0.0,
                                  update='sgd', learning_rate_decay=1,
                                  sample_batches = False,
                                  learning_rate=1e-3, verbose=True)
```

The above code:

- take the first 20 examples from CIFAR-10
- turn off regularization (reg = 0.0)
- use simple vanilla 'sgd'

details:

- (learning_rate_decay = 1 means no decay, the learning rate will stay constant)
- sample_batches = False means we're doing full gradient descent, not mini-batch SGD
- we'll perform 200 updates (epochs = 200)

"**epoch**": number of times we see the training set

Training neural network

Lets try to train now…

Tip: Make sure that you can overfit very small portion of the data

```
model = init_two_layer_model(32*32*3, 50, 10) # input size, hidden size, number of classes
trainer = ClassifierTrainer()
X_tiny = X_train[:20] # take 20 examples
y_tiny = y_train[:20]
best_model, stats = trainer.train(X_tiny, y_tiny, X_tiny, y_tiny,
                                  model, two_layer_net,
                                  num_epochs=200, reg=0.0,
                                  update='sgd', learning_rate_decay=1,
                                  sample_batches = False,
                                  learning_rate=1e-3, verbose=True)
```

```
Finished epoch 1 / 200: cost 2.302603, train: 0.400000, val 0.400000, lr 1.000000e-03
Finished epoch 2 / 200: cost 2.302258, train: 0.450000, val 0.450000, lr 1.000000e-03
Finished epoch 3 / 200: cost 2.301849, train: 0.600000, val 0.600000, lr 1.000000e-03
Finished epoch 4 / 200: cost 2.301196, train: 0.650000, val 0.650000, lr 1.000000e-03
Finished epoch 5 / 200: cost 2.300044, train: 0.650000, val 0.650000, lr 1.000000e-03
Finished epoch 6 / 200: cost 2.297864, train: 0.550000, val 0.550000, lr 1.000000e-03
Finished epoch 7 / 200: cost 2.293595, train: 0.600000, val 0.600000, lr 1.000000e-03
Finished epoch 8 / 200: cost 2.285096, train: 0.550000, val 0.550000, lr 1.000000e-03
Finished epoch 9 / 200: cost 2.268094, train: 0.550000, val 0.550000, lr 1.000000e-03
Finished epoch 10 / 200: cost 2.234787, train: 0.500000, val 0.500000, lr 1.000000e-03
Finished epoch 11 / 200: cost 2.173187, train: 0.500000, val 0.500000, lr 1.000000e-03
Finished epoch 12 / 200: cost 2.076862, train: 0.500000, val 0.500000, lr 1.000000e-03
Finished epoch 13 / 200: cost 1.974090, train: 0.400000, val 0.400000, lr 1.000000e-03
Finished epoch 14 / 200: cost 1.895885, train: 0.400000, val 0.400000, lr 1.000000e-03
Finished epoch 15 / 200: cost 1.820876, train: 0.450000, val 0.450000, lr 1.000000e-03
Finished epoch 16 / 200: cost 1.737430, train: 0.450000, val 0.450000, lr 1.000000e-03
Finished epoch 17 / 200: cost 1.642356, train: 0.500000, val 0.500000, lr 1.000000e-03
Finished epoch 18 / 200: cost 1.535239, train: 0.600000, val 0.600000, lr 1.000000e-03
Finished epoch 19 / 200: cost 1.421527, train: 0.600000, val 0.600000, lr 1.000000e-03
```

```
Finished epoch 195 / 200: cost 0.002694, train: 1.000000, val 1.000000, lr 1.000000e-03
Finished epoch 196 / 200: cost 0.002674, train: 1.000000, val 1.000000, lr 1.000000e-03
Finished epoch 197 / 200: cost 0.002655, train: 1.000000, val 1.000000, lr 1.000000e-03
Finished epoch 198 / 200: cost 0.002635, train: 1.000000, val 1.000000, lr 1.000000e-03
Finished epoch 199 / 200: cost 0.002617, train: 1.000000, val 1.000000, lr 1.000000e-03
Finished epoch 200 / 200: cost 0.002597, train: 1.000000, val 1.000000, lr 1.000000e-03
finished optimization. best validation accuracy: 1.000000
```

Very small loss, train accuracy 1.00, nice!

Training neural network

Lets try to train now…

I like to start with small regularization and find learning rate that makes the loss go down.

loss not going down:
learning rate too low
loss exploding:
learning rate too high

```
model = init_two_layer_model(32*32*3, 50, 10) # input size, hidden size, number of classes
trainer = ClassifierTrainer()
best_model, stats = trainer.train(X_train, y_train, X_val, y_val,
                                  model, two_layer_net,
                                  num_epochs=10, reg=0.000001,
                                  update='sgd', learning_rate_decay=1,
                                  sample_batches = True,
                                  learning_rate=1e-6, verbose=True)
```

Training neural network

Lets try to train now…

I like to start with small regularization and find learning rate that makes the loss go down.

loss not going down:
learning rate too low
loss exploding:
learning rate too high

```
model = init_two_layer_model(32*32*3, 50, 10) # input size, hidden size, number of classes
trainer = ClassifierTrainer()
best_model, stats = trainer.train(X_train, y_train, X_val, y_val,
                                  model, two_layer_net,
                                  num_epochs=10, reg=0.000001,
                                  update='sgd', learning_rate_decay=1,
                                  sample_batches = True,
                                  learning_rate=1e-6, verbose=True)
```

```
Finished epoch 1 / 10: cost 2.302576, train: 0.080000, val 0.103000, lr 1.000000e-06
Finished epoch 2 / 10: cost 2.302582, train: 0.121000, val 0.124000, lr 1.000000e-06
Finished epoch 3 / 10: cost 2.302558, train: 0.119000, val 0.138000, lr 1.000000e-06
Finished epoch 4 / 10: cost 2.302519, train: 0.127000, val 0.151000, lr 1.000000e-06
Finished epoch 5 / 10: cost 2.302517, train: 0.158000, val 0.171000, lr 1.000000e-06
Finished epoch 6 / 10: cost 2.302518, train: 0.179000, val 0.172000, lr 1.000000e-06
Finished epoch 7 / 10: cost 2.302466, train: 0.180000, val 0.176000, lr 1.000000e-06
Finished epoch 8 / 10: cost 2.302452, train: 0.175000, val 0.185000, lr 1.000000e-06
Finished epoch 9 / 10: cost 2.302459, train: 0.206000, val 0.192000, lr 1.000000e-06
Finished epoch 10 / 10: cost 2.302420, train: 0.190000, val 0.192000, lr 1.000000e-06
finished optimization. best validation accuracy: 0.192000
```

Loss barely changing: Learning rate must be too low. (could also be reg too high)

Notice train/val accuracy goes to 20% though, what's up with that? (remember this is softmax)

Training neural network

Lets try to train now…

I like to start with small regularization and find learning rate that makes the loss go down.

loss not going down:
learning rate too low
loss exploding:
learning rate too high

```
model = init_two_layer_model(32*32*3, 50, 10) # input size, hidden size, number of classes
trainer = ClassifierTrainer()
best_model, stats = trainer.train(X_train, y_train, X_val, y_val,
                                  model, two_layer_net,
                                  num_epochs=10, reg=0.000001,
                                  update='sgd', learning_rate_decay=1,
                                  sample_batches = True,
                                  learning_rate=1e6, verbose=True)
```

Okay now lets try learning rate 1e6. What could possibly go wrong?

Training neural network

Lets try to train now…

I like to start with small regularization and find learning rate that makes the loss go down.

```
model = init_two_layer_model(32*32*3, 50, 10) # input size, hidden size, number of classes
trainer = ClassifierTrainer()
best_model, stats = trainer.train(X_train, y_train, X_val, y_val,
                                  model, two_layer_net,
                                  num_epochs=10, reg=0.000001,
                                  update='sgd', learning_rate_decay=1,
                                  sample_batches = True,
                                  learning_rate=1e6, verbose=True)
```

```
/home/karpathy/cs231n/code/cs231n/classifiers/neural_net.py:50: RuntimeWarning: divide by zero en
countered in log
  data_loss = -np.sum(np.log(probs[range(N), y])) / N
/home/karpathy/cs231n/code/cs231n/classifiers/neural_net.py:48: RuntimeWarning: invalid value enc
ountered in subtract
  probs = np.exp(scores - np.max(scores, axis=1, keepdims=True))
Finished epoch 1 / 10: cost nan, train: 0.091000, val 0.087000, lr 1.000000e+06
Finished epoch 2 / 10: cost nan, train: 0.095000, val 0.087000, lr 1.000000e+06
Finished epoch 3 / 10: cost nan, train: 0.100000, val 0.087000, lr 1.000000e+06
```

cost: NaN almost always means high learning rate...

loss not going down:
learning rate too low
loss exploding:
learning rate too high

Training neural network

Lets try to train now…

I like to start with small regularization and find learning rate that makes the loss go down.

loss not going down:
learning rate too low
loss exploding:
learning rate too high

```
model = init_two_layer_model(32*32*3, 50, 10) # input size, hidden size, number of classes
trainer = ClassifierTrainer()
best_model, stats = trainer.train(X_train, y_train, X_val, y_val,
                                  model, two_layer_net,
                                  num_epochs=10, reg=0.000001,
                                  update='sgd', learning_rate_decay=1,
                                  sample_batches = True,
                                  learning_rate=3e-3, verbose=True)
```

```
Finished epoch 1 / 10: cost 2.186654, train: 0.308000, val 0.306000, lr 3.000000e-03
Finished epoch 2 / 10: cost 2.176230, train: 0.330000, val 0.350000, lr 3.000000e-03
Finished epoch 3 / 10: cost 1.942257, train: 0.376000, val 0.352000, lr 3.000000e-03
Finished epoch 4 / 10: cost 1.827868, train: 0.329000, val 0.310000, lr 3.000000e-03
Finished epoch 5 / 10: cost inf, train: 0.128000, val 0.128000, lr 3.000000e-03
Finished epoch 6 / 10: cost inf, train: 0.144000, val 0.147000, lr 3.000000e-03
```

3e-3 is still too high. Cost explodes….

=> Rough range for learning rate we should be cross-validating is somewhere [1e-3 … 1e-5]

Cross-validation strategy

I like to do **coarse -> fine** cross-validation in stages

First stage: only a few epochs to get rough idea of what params work
Second stage: longer running time, finer search
… (repeat as necessary)

Tip for detecting explosions in the solver:
If the cost is ever > 3 * original cost, break out early

Cross-validation strategy (cont'd)

For example: run coarse search for 5 epochs

```
max_count = 100
for count in xrange(max_count):
      reg = 10**uniform(-5, 5)
      lr = 10**uniform(-3, -6)

      trainer = ClassifierTrainer()
      model = init_two_layer_model(32*32*3, 50, 10) # input size, hidden size, number of classes
      trainer = ClassifierTrainer()
      best_model_local, stats = trainer.train(X_train, y_train, X_val, y_val,
                                              model, two_layer_net,
                                              num_epochs=5, reg=reg,
                                              update='momentum', learning_rate_decay=0.9,
                                              sample_batches = True, batch_size = 100,
                                              learning_rate=lr, verbose=False)
```

note it's best to optimize in log space

```
val_acc: 0.412000, lr: 1.405206e-04, reg: 4.793564e-01, (1 / 100)
val_acc: 0.214000, lr: 7.231888e-06, reg: 2.321281e-04, (2 / 100)
val_acc: 0.208000, lr: 2.119571e-06, reg: 8.011857e+01, (3 / 100)
val_acc: 0.196000, lr: 1.551131e-05, reg: 4.374936e-05, (4 / 100)
val_acc: 0.079000, lr: 1.753300e-05, reg: 1.200424e+03, (5 / 100)
val_acc: 0.223000, lr: 4.215128e-05, reg: 4.196174e+01, (6 / 100)
val_acc: 0.441000, lr: 1.750259e-04, reg: 2.110807e-04, (7 / 100)
val_acc: 0.241000, lr: 6.749231e-05, reg: 4.226413e+01, (8 / 100)
val_acc: 0.482000, lr: 4.296863e-04, reg: 6.642555e-01, (9 / 100)
val_acc: 0.079000, lr: 5.401602e-06, reg: 1.599828e+04, (10 / 100)
val_acc: 0.154000, lr: 1.618508e-06, reg: 4.925252e-01, (11 / 100)
```

nice

Cross-validation strategy (cont'd)

Now run finer search...

```
max_count = 100
for count in xrange(max_count):
      reg = 10**uniform(-5, 5)
      lr = 10**uniform(-3, -6)
```

adjust range →

```
max_count = 100
for count in xrange(max_count):
      reg = 10**uniform(-4, 0)
      lr = 10**uniform(-3, -4)
```

```
val_acc: 0.527000, lr: 5.340517e-04, reg: 4.097824e-01, (0 / 100)
val_acc: 0.492000, lr: 2.279484e-04, reg: 9.991345e-04, (1 / 100)
val_acc: 0.512000, lr: 8.680827e-04, reg: 1.349727e-02, (2 / 100)
val_acc: 0.461000, lr: 1.028377e-04, reg: 1.220193e-02, (3 / 100)
val_acc: 0.460000, lr: 1.113730e-04, reg: 5.244309e-02, (4 / 100)
val_acc: 0.498000, lr: 9.477776e-04, reg: 2.001293e-03, (5 / 100)
val_acc: 0.469000, lr: 1.484369e-04, reg: 4.328313e-01, (6 / 100)
val_acc: 0.522000, lr: 5.586261e-04, reg: 2.312685e-04, (7 / 100)
val_acc: 0.530000, lr: 5.808183e-04, reg: 8.259964e-02, (8 / 100)
val_acc: 0.489000, lr: 1.979168e-04, reg: 1.010889e-04, (9 / 100)
val_acc: 0.490000, lr: 2.036031e-04, reg: 2.406271e-03, (10 / 100)
val_acc: 0.475000, lr: 2.021162e-04, reg: 2.287807e-01, (11 / 100)
val_acc: 0.460000, lr: 1.135527e-04, reg: 3.905040e-02, (12 / 100)
val_acc: 0.515000, lr: 6.947668e-04, reg: 1.562808e-02, (13 / 100)
val_acc: 0.531000, lr: 9.471549e-04, reg: 1.433895e-03, (14 / 100)
val_acc: 0.509000, lr: 3.140888e-04, reg: 2.857518e-01, (15 / 100)
val_acc: 0.514000, lr: 6.438349e-04, reg: 3.033781e-01, (16 / 100)
val_acc: 0.502000, lr: 3.921784e-04, reg: 2.707126e-04, (17 / 100)
val_acc: 0.509000, lr: 9.752279e-04, reg: 2.850865e-03, (18 / 100)
val_acc: 0.500000, lr: 2.412048e-04, reg: 4.997821e-04, (19 / 100)
val_acc: 0.466000, lr: 1.319314e-04, reg: 1.189915e-02, (20 / 100)
val_acc: 0.516000, lr: 8.039527e-04, reg: 1.528291e-02, (21 / 100)
```

53% - relatively good for a 2-layer neural net with 50 hidden neurons.

Validation

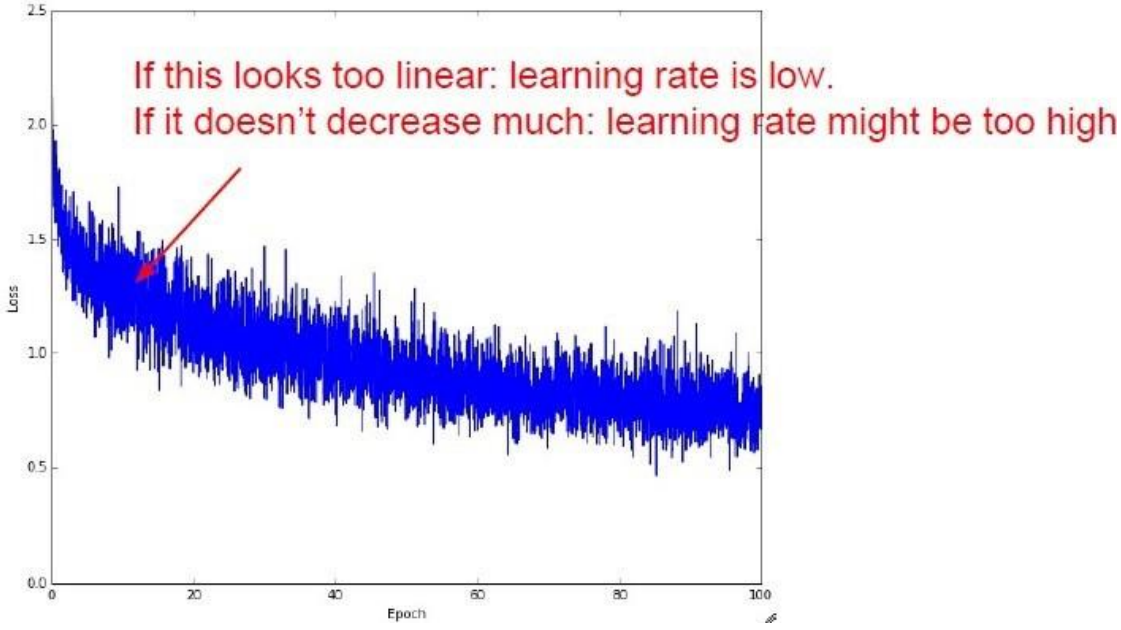
Normally you can't afford a huge computational budget for expensive cross-validations.
Need to rely more on intuitions and visualizations…

Visualizations to play with:

- **loss** function
- validation and training **accuracy**
- min,max,std for **values and updates**, (and monitor their ratio)
- **first-layer visualization** of weights (if working with images)

Validation

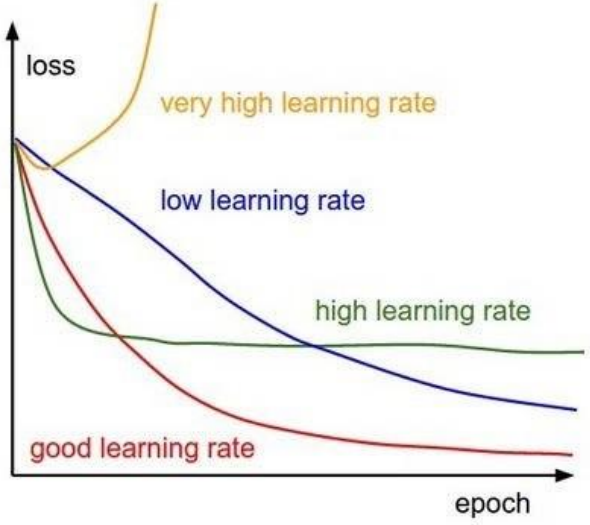
Monitor and visualize the loss curve

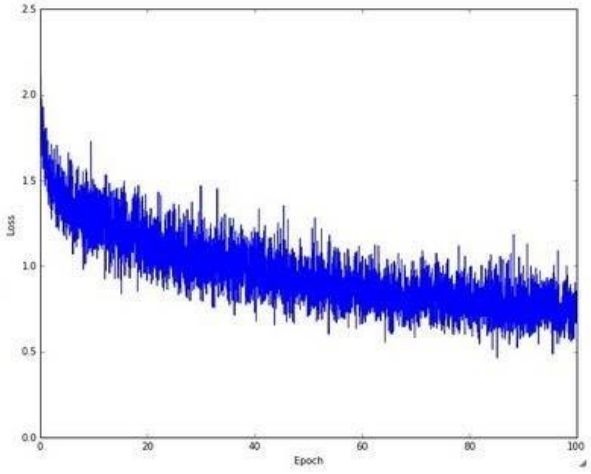


Validation

- Choosing learning rates

LeCun, Yann A., et al. "Efficient backprop." Neural networks: Tricks of the trade. Springer Berlin Heidelberg, 2012. 9-48.

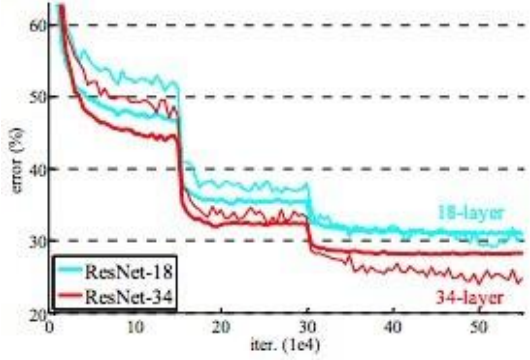




Learning rates and updates

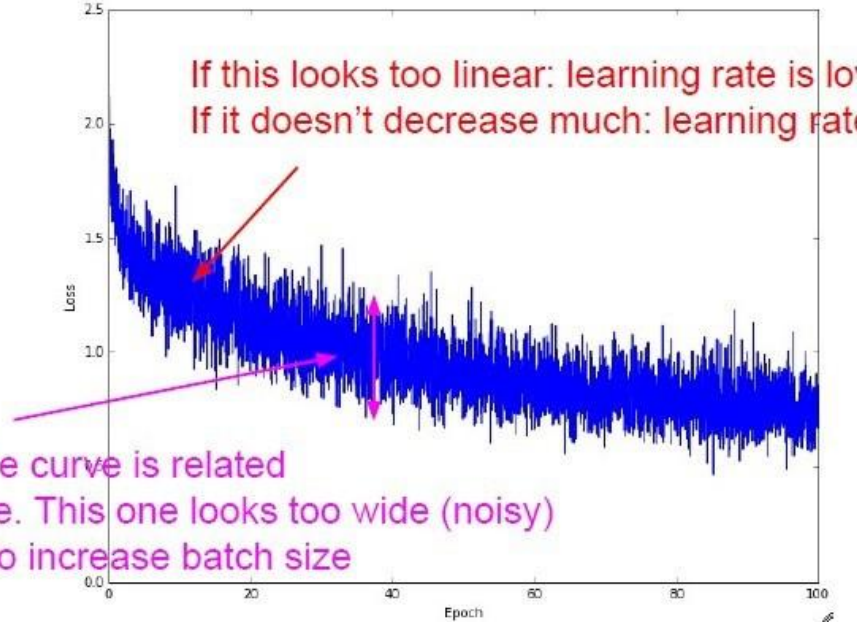
- SGD + Momentum > SGD
- Momentum 0.9 usually works well
- <u>Decrease the learning rate over time</u>
  (people use 1/t, exp(-t), or steps)

  simplest: learning_rate *= 0.97 every epoch (or so)



Validation

Monitor and visualize the loss curve

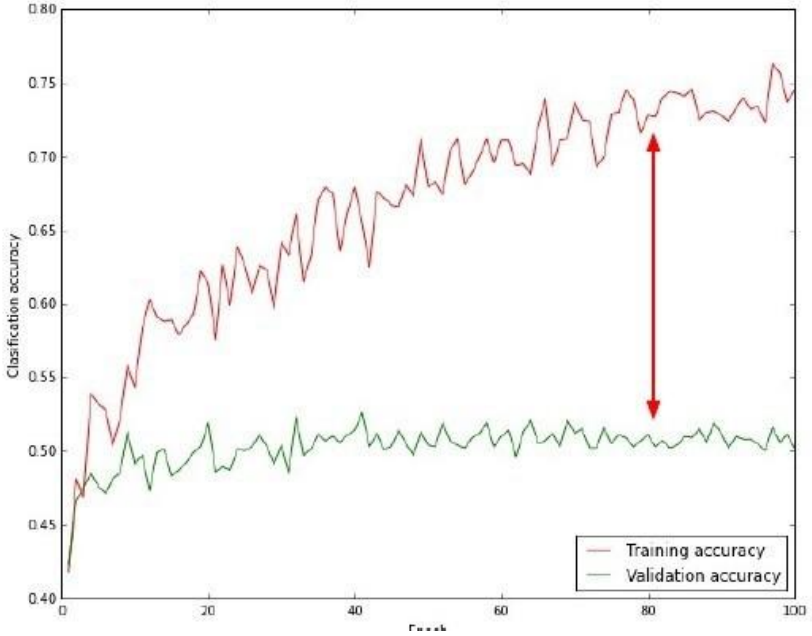


If this looks too linear: learning rate is low.
If it doesn't decrease much: learning rate might be too high

the "width" of the curve is related
to the batch size. This one looks too wide (noisy)
=> might want to increase batch size

Validation

Monitor and visualize the accuracy:



big gap = overfitting

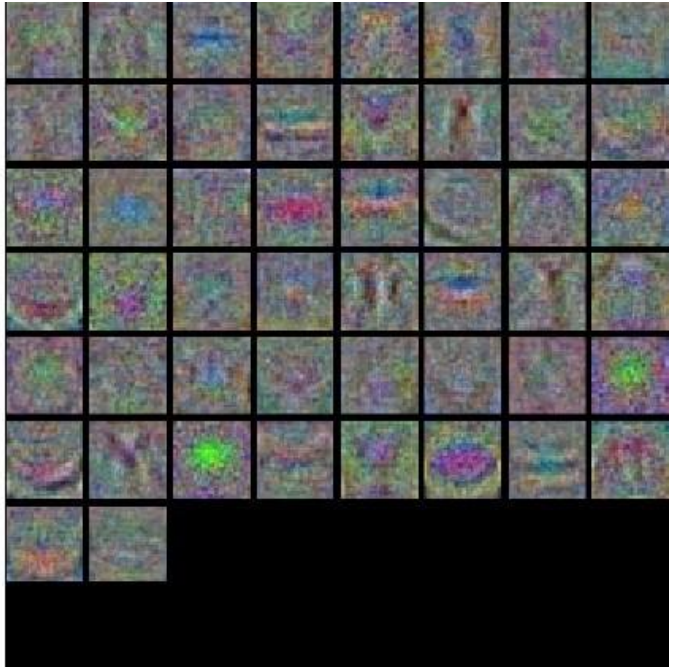
=> increase regularization strength

no gap

=> increase model capacity

Validation

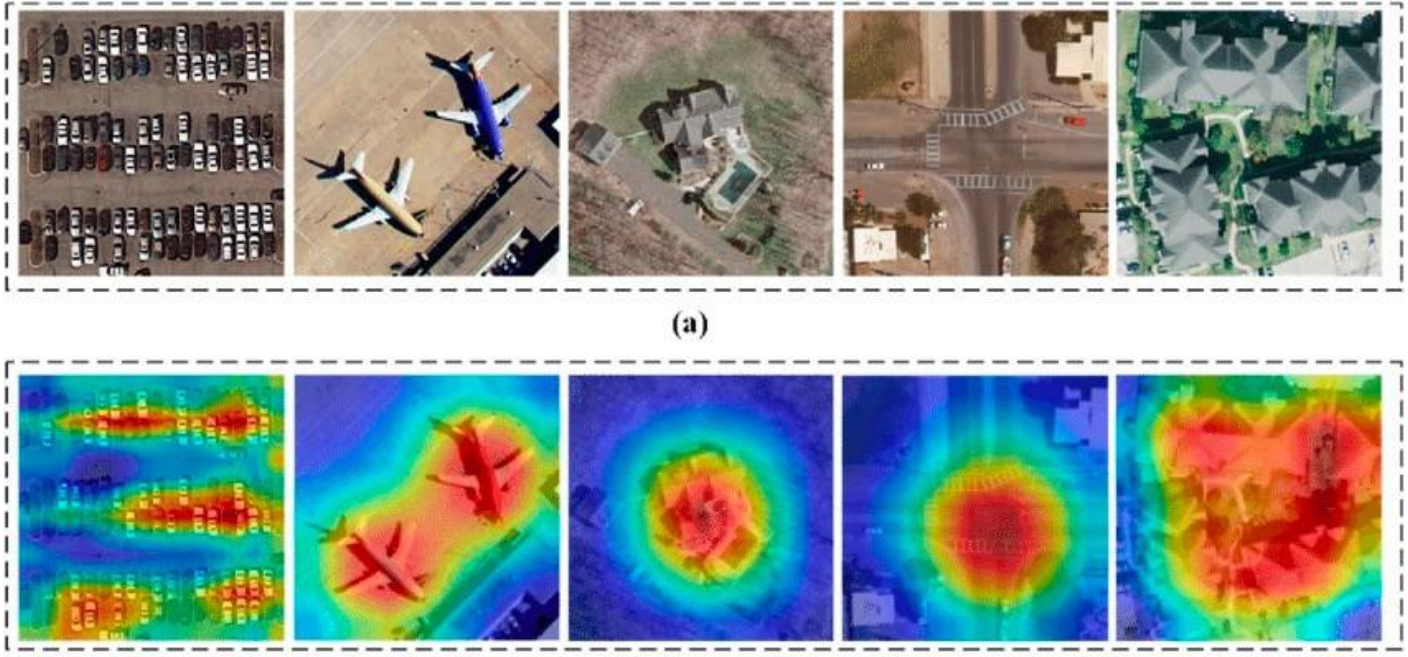
Visualizing first-layer weights:



Noisy weights => Regularization maybe not strong enough

Validation

- GradCAM is one of best tools to visualize deep neural networks.



(a)

(b)

Inference (Test time)

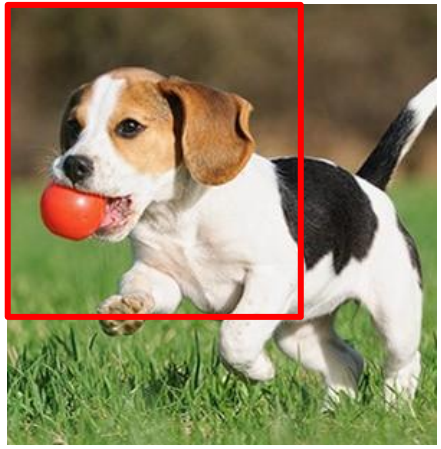
Model Ensembles

- One way to *always* improve final accuracy: take several trained models and average their predictions

Center crop / 10 crop inference

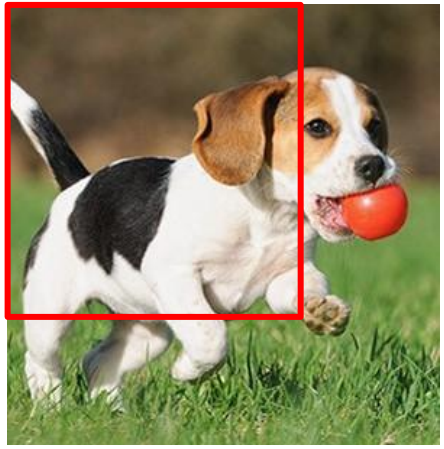
224

224



224

224



Conclusion (Remind)

- **I am sure that you can answer following questions if you fully understood this material.**

1. What is a process of training CNN?
2. What is Batch normalization?
3. What kind of optimizer will you use?
4. What is data augmentation?
5. What is model ensemble?

Thank You!